

Binary

Base Conversions		
$\begin{aligned} &0\ 1101_2 \\ &= (1 \times 2^0 + 0 \times 2^1 + 1 \times 2^2 + 1 \times 2^3)_{10} \\ &= (1 \times 1 + 0 \times 2 + 1 \times 4 + 1 \times 8)_{10} \\ &= 13_{10} \end{aligned}$	$\begin{aligned} 13_{10} &= 6_{10} \times 2_{10} + 1_{10} \\ 6_{10} &= 3_{10} \times 2_{10} + 0_{10} \\ 3_{10} &= 1_{10} \times 2_{10} + 1_{10} \\ 1_{10} &= 0_{10} \times 2_{10} + 1_{10} \end{aligned}$	$\begin{aligned} &13_{10} \\ &= 8_{10} + 4_{10} + 1_{10} \\ &= (1000_2 + 100_2 + 1_2) \\ &= 0\ 1101_2 \end{aligned}$

Fractions		Addition and Subtraction
$\begin{aligned} &0\ 1010\ ^\wedge\ 0101_2 = 10\ 5/16 \\ &\text{Whole Number: } 2+8=10 \text{ (value before } ^\wedge\text{)} \\ &\text{Fraction:} \\ &\quad \text{Denominator: } 2^4 \text{ (digits after } ^\wedge\text{)} \\ &\quad \text{Numerator: } 1+4=5 \text{ (value after } ^\wedge\text{)} \end{aligned}$	$\begin{aligned} &0.8125 = 0\ ^\wedge\ 1101_2 \\ &0.8125 \times 2 = 1 + 0.625 \\ &0.625 \times 2 = 1 + 0.25 \\ &0.25 \times 2 = 0 + 0.5 \\ &0.5 \times 2 = 1 \end{aligned}$	Same as decimal numbers but base 2. ex. $\begin{array}{r} 1\ 1\ 0\ 1\ 0\ 1\ 0 \\ -\quad 1\ 1\ 0\ 1\ 1\ 1 \\ =\quad 1\ 1\ 0\ 0\ 1\ 1 \end{array}$

Negative and Positive Values	
Converting	Indicating
Swap all digits on the left side of the right-most 1. ex. 0 1010 ^ 0101 converts into 1 0101 ^ 1011	Positive: Left-Most Digit=0 Negative: Left-Most Digit=1
※ Always be aware of the sign! ※ Converting Negative Binary Fractions into Decimal Fractions: Neg_Bin_Frac → Pos_Bin_Frac → Pos_Dec_Frac → Neg_Dec_Frac	

Hexadecimal

Base Conversions		Addition and Subtraction
00 0000 0 01 0001 1 02 0010 2 03 0011 3 04 0100 4 05 0101 5 06 0110 6 07 0111 7 08 1000 8 09 1001 9 10 1010 A 11 1011 B 12 1100 C 13 1101 D 14 1110 E 15 1111 F	$\begin{aligned} &A_{16} = 1010_2 \quad C_{16} = 1100_2 \quad 0_{16} = 0000_2 \\ &\Rightarrow AC0_{16} = 0\ 1010\ 1100\ 0000_2 \\ &1010_2 = A_{16} \quad 1100_2 = C_{16} \quad 0000_2 = 0_{16} \\ &\Rightarrow 0\ 1010\ 1100\ 0000_2 = AC0_{16} \\ &AC0_{16} \\ &= (0 \times 16^0 + 12 \times 16^1 + 10 \times 16^2)_{10} \\ &= (0 \times 1 + 12 \times 16 + 10 \times 256)_{10} \\ &= 2752_{10} \\ &2752_{10} = 172_{10} \times 16_{10} + 0_{10} \times 16_{10} \quad \text{aka } 0_{16} \\ &172_{10} = 10_{10} \times 16_{10} + 0.75_{10} \times 16_{10} \quad \text{aka } C_{16} \\ &10_{10} = 0_{10} \times 16_{10} + 0.625_{10} \times 16_{10} \quad \text{aka } A_{16} \end{aligned}$	Same as decimal numbers but base 16. ex. $\begin{array}{r} D\ 8\ 8\ 1 \\ -\quad 8\ 8\ C\ C \\ =\quad 4\ F\ B\ 5 \end{array}$

ASCII (American Standard Code Information Interchange)

Conversions		
(ABCDE FGHIJ KLMNO PQRST UVWXY Z) (abcde fg hij klmno pqrst uvwxy z)		
Letter	Position + Property	Representation
K	11 ₁₀ (aka B_{16}) + Upper (aka 40 ₁₆)	4B
k	11 ₁₀ (aka B_{16}) + Lower (aka 60 ₁₆)	6B

ASCII TABLE

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	1	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	2	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	3	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	4	[END OF TRANSMISSION]	36	24	\$	68	44	D	100	64	d
5	5	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	6	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	7	[BELL]	39	27	'	71	47	G	103	67	g
8	8	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	9	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[ENG OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]

How Real Numbers are Stored

For example, (1 00000111 0011001101) is how (− 102 3/8) is stored.		
The Number:	1 ^ 0011 0011 01	(Negative)
Converted:	0 ^ 1100 1100 11	(Positive)
Power:	0000 0111	(Move Binary Point +7 Digits)
Result:	0110 0110 ^ 011	(+ 102 3/8)
⇒ Answer: (1 00000111 0011001101) is how (− 102 3/8) is stored.		

Coding Terms

Term	Explanation		Term	Explanation
Syntax	The grammar in coding.		Comment	Notes that are ignored in execution.
Statement	Instructions that are to be runned.		Expression	A claim that may be true or false.
Function	A piece of named code waiting to be called by programmers (to carry out a series of actions).			
Library	A module where non-elementary functions are categorized and stored in.			
Algorithm	An algorithm is a set of steps that describe how to solve a problem or perform a task.			

Languages

	Python	Other Languages
Significance	Similar to English and Programmer-Friendly	Basic and Interpreter-Friendly
Syntax	Spaces, Indentations, and Lines	{Semi-columns, (Brackets), and Parenthesis};
Comment	# After Pound Key	/* Between Star Keys and Slashes */
Variables	Flexible Data Type (ex. albert = 17) No Declaration Required	Inflexible Data Type (ex. int albert = 17) Declaration of Variable Required (At Beginning)

Organization of Programs

Header Library imports Define Constants Define and initialize variables Input statements Processing statements Output statements	Organizing a program into functional (by usage) sections helps for future readability and maintenance.	Header (A Common Industry Convention): A multi-line comment including name of program,name of developer, finishing date, purpose of the program (short description), and a history of maintenance.
--	--	---

Data Types

Variables				
str()	(String, Text) such as “5” or ‘5’ (but not “5”)		Naming Conventions	
int()	(Integer, Numeric) such as 5 ※int(0101, 2) returns 5		Camel Case	eg. albertHong
float()	(Float, Numeric) such as 5.0		Pascal Case	eg. AlbertHong
bool()	(Boolean) such as True, False, or (5 >= 0) which means True		Snake Case	eg. my_name or My_Name
※Start: Alphabetic Character or Underscore (but Not Numbers) ※Components: Alphabetic Characters, Numeric Characters, or Underscore ※Case-Sensitive: Cases (Upper or Lower) Matters				

Mathematical Symbols and Expressions

Arithmetics	Comparison Operators	Logical Operators
+ − ∗ / % ∗∗ // Addition, Subtraction, Multiplication, Division, Modulus (Remainder), Exponential (Power), Floor Division (Integer Division)	== != > < >= <= Equal, Equal Not, Greater, Lesser, Not Lesser, Not Greater	(...) and (...) (...) or (...) not(...)
	※ Commonly Seen in If-Statements and Loops	
※ Number + Number (Adding) ※ String + String (Combining Texts)	※ Assignment Operator (=) Verb ※ Comparison Operator (==) Adjective	※ not(A or B) not(A) and not(B)

Elementary Functions

Functions	Features and Reminders
print(“Hello World”) print(“Hello” + str(1.0) + “World\n”)	Display text or variable values (or combined) on screen. (, A Space Between) (+ No Spaces Between Strings) (\n Go To Next Line)
albert = input(“Enter Albert’s Age: ”)	Prompt user to enter something. (The prompt will be printed out.)
albert = “17”	Define variables. Variables must be defined (i.e. assigned a value).
albert = int(albert)	Cast variables to different data types.
type(albert) returns <class ‘int’>	Determine the data type of a variable.
albert = albert + 1	Modify the value of variables. (Combining strings or calculating numbers.)
round(3.3333, 2) returns 3.33	Round a number to whatever (default 0) decimal points.

If-Level

	Syntax	Reminders
if	if (expression): (codeblock)	※ If an expression is true, the corresponding code-block will be runned. Other statements not indented do not belong to the if-level and will be runned regardless of the expressions. ※ All of the previous conditions in the if-level must not be true to have elif-code-block runned (even if the expression is true). Be aware of the order. ※ An if-code-block starts a new if-level. Use the elif-code-block so that all mentioned conditions are in the same if-level and considered before the else-code-block is runned.
else if	elif (expression): (codeblock)	
else	else: (codeblock)	

(Pseudo-)Random Numbers

Syntax	Reminders
import random x = random.randrange(start, stop)	※ Must imported random library to call out specific functions from later on. ※ The stop number will never be selected, but the start number (default 0) will.
※ (Application) Probability: Use the random function to do a lottery in the computer.	

While-Levels

Example		Reminders
<pre> grade = 20 while (grade < 100): grade = grade + 10 print(grade) if not(grade >= 60): continue print("High Enough") if (grade == 90): break else: print("Full Mark") print("Grade:", grade) </pre>	30	while Do something while the expression is true. ※Phrase “while True” (without other expressions) means “always”.
	40	continue Continue on the next run of the loop immediately. ※Statements in the while-code-block after “continue” will not be conducted when “continue” is conducted.
	50	
	60	else Do something else when the expression becomes false.
	High Enough	
	70	break Break the entire while-level. ※Statements in the else-code-block will not be conducted when “break” is conducted (but statements outside the while-level will).
	High Enough	
	80	
	High Enough	
	90	
	High Enough	
	Grade: 90	

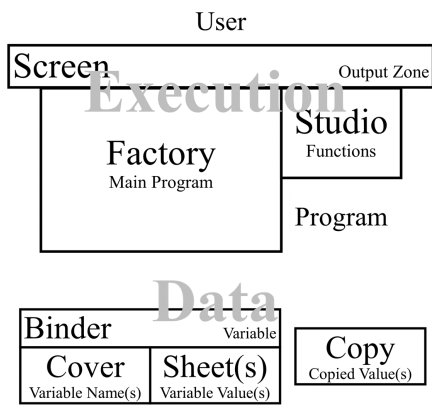
For-Levels

Syntax and Reminders		
for	for i in range(start, stop, increment): (codeblock)	※start: default 0 ※increment: default 1
	※Run code block multiple times, having the first time (i) equals (start), the second time (i) equals (start + increment), the third time (i) equals (start + 2increment),, until (i) equals (stop). E.G. Code– for i in range(5): –runs 5 times with (i) being 0, 1, 2, 3, 4. ※The range function can also be used in expressions. E.G. (2 in range(5)) returns True	
	else Do something else when (i) equals (stop). continue ※Other reminders and syntax are similar to while-levels. break ※Nested Loops: Loop inside Loop (Apply for Both While-Loops and For-Loops)	* ** *** **** *****
※ASCII art: creating pictures by printing out ASCII characters with for-levels.		

Functions Define

A key concept in programming is modularity: writing code in smaller, reusable chunks. Programs can be shortened by the application of the modular or reuse concept–define functions at the beginning and call them out later on (instead of writing the code block again and again whenever there is a need).	
Syntax	Variables
<pre> def the_function(a, b, c): (codeblock containing abc) (other code) the_function(7, 8, 9) </pre>	※Parameter: variable specified in the brackets (as many as you wish with commas). ※Argument: value assigned to the parameters when the function is called.
※A function can: do a task, do a task involving the arguments, or return a value according to the arguments. ※To return a value: write a return-statement using the code “return” in the code block. E.G. return a+b+c ※Parameters can have default values. E.G. def the_function(a=1, b=2, c=3): ※Parameters can be assigned values using keyword arguments. E.G. the_function(c=9, b=8, a=7)	

Scope (Lifetime) of Variables



- ※ Imagine that functions are studios beside a factory. When called, they can take files containing information from the factory and carry out specific tasks.
- ※ Variables in function definitions other than the parameters shall be defined inside the definition, and they exist only in the definition. (Only the parameters are transferred into the studio, so other variables from the factory cannot be reached. Samely, when functions are completed, only the returned values are delivered to the factory, other variables are forgotten and become undefined in status.)
- ※ Pass by Value: Taking a copy of information as a material for further activities. In other words, any modification done to the copy does not affect the actual value stored in the variable. Scalar variables pass by value in functions.
- ※ Pass by Reference: Taking the item that contains the information itself. To say “pass by reference,” the binder is what you are dealing with. The binder goes from the factory to the studio, modified, and goes back to the factory (no matter if the function is returning it). Collection variables pass by reference in functions.

Data Types (Collections)

Collections (Containers)–Summary

A (scalar) variable can only be assigned one single value, which becomes infeasible when there are multiple or an unknown amount of values to store. The data structure that handles this situation is the array. However, we seldom use arrays in Python, but different types of collections. You can store one type of data only in an array (for example, an all-string-array), whereas collections are able to store a combination of all data types. Collections are memory allocations where elements are stored, which should also be defined before being called.

Examples	List: [0, 6, 2, 5] Tuple: (0, 6, 2, 5) Set: { 0, 6, 2, 5 } Dictionary: { “M”:(0,6), “D”:(2,5) }			
List	Ordered	Indexed (Indices)	Mutable	Duplicate-Entries-Allowed
Tuple	Ordered	Indexed (Indices)	Immutable	Duplicate-Entries-Allowed
Set	Unordered	Unindexed	Immutable	No-Duplicate-Entries
Dictionary	Unordered	Labelled (Keys)	Mutable	No-Duplicate-Entries

Dictionaries contain elements of data pairs in the { key : value, key : value, key : value ... } form. Usually, when we mention a dictionary in the code, we are referring to the keys instead of the values.

Unordered collections would display elements in random order when printed out.

Immutable objects can still be edited using concatenation or special functions. The difference is the effort: when editing immutable objects, the idea is to make a copy, paste it alongside with another value, and delete the original version—you need an assignment operator (=). When editing mutable objects, elements change in place; that is, no “=” in the code.

No-Duplicate-Entries means if a specific value (or key for dictionary) appear more than once, only the last one counts.

Iterables are the data types that can be applied in for-loops, including strings, collections, and the range() function.

E.G. for i in iterable: with (i) being the 0th, 1st, 2nd, 3rd, ..., to the last element (pass by value).

Ideas Regarding Collections

Expression Collections can also be used in expressions.

E.G. (“cat” in animal_list) may return True or False

Dimension Elements in collections can also be collections. For example, a list of tuples [(0,6), (2,5)] can be seen as a two-dimension-collection. The index would be written in form collection[a][b].

Collection Methods (Tasks Associated with Objects)

In the following examples, haha and hehe are the name of a collection. Also, note that these methods do not change the dimension of haha. (Noticing the difference between [0, 6, 2, 5] and [0, 6, [2, 5]].)

Method	Syntax	Reminders	Application
Type	type(haha)	※Return type of collection.	All
Length	len(haha)	※Counts up from 1 (with regards to for-loops).	All
Access	haha[key]	※Indices of the elements count up from 0. ※Indices can also be negative. For example, the last element in haha has an index of (-1)	Dictionaries
	haha[index]		Indexed
Access (Slice)	haha[start : stop : step]	※Slices include start but not stop. ※Slice default first to last element inclusive. ※Step default 1. ※No errors when index out of range. It goes as far as possible and stops when no more items.	
Delete	del haha del haha[i] del haha[start : stop : step]	※Deleted collection becomes undefined. ※Elements behind move forward after one or more elements are deleted.	All Indexed or Labelled Indexed
Remove	haha.remove("A")	※Remove first element of that value.	Lists and Sets
Pop	haha.pop(5)	※Remove item by index (default -1).	Mutable
Extend	haha.extend(hehe)	※The second collection, hehe, does not need to be a list. The result will be a list.	Lists
Concatenate	haha = haha + ["A",]	※Brackets and commas avoid arithmetics and tell program if you are adding elements to haha or adding a collection as an element in haha.	Square Brackets for Lists and Round Brackets for Tuples
Maximum	max(haha)	※Return biggest value (or key for dictionaries).	All
Minimum	min(haha)	※Return smallest value (or key for dictionary).	All
Count	haha.count("A")	※Count how many times value appear in list.	Lists, Tuples and Strings
Sort	haha.sort(key, reverse)	※Key: sorting reference. For example, def k(x): or ...key = lambda x:x[1]... return x[1] ...key = k... “For each element x in haha, we are sorting by looking at x[1].” ※Reverse: True for Descend, False for Ascend (Default False)	Lists
	hehe = sorted(haha, key, reverse)	※Creates hehe as a list. ※When haha is a dictionary, the sorted() function converts it into a list of tuples (with key and value being x[0] and x[1]) and then sort it.	All

Other Methods: Search it up on Google! (Or look for it on the python-website:

<https://docs.python.org/zh-tw/3/tutorial/datastructures.html> (Traditional Chinese)

<https://docs.python.org/3/tutorial/datastructures.html> (English)

Lists		
Create	<pre>list = [0, 6, 2, 5] list = string.split(" ") list = list_1 + list_2 list = other_list * 2 list = [formula for i in range()] list = [formula for i in iterable if expression] this_list = that_list</pre>	※ Create list with [,] square brackets and commas. ※ By splitting a string, you get a list. ※ Does not change dimension. ※ Still does not change dimension. ※ Create list from numbers using comprehensions. ※ Create list from an iterable using comprehensions. ※ Duplicate lists. (The only condition passing by reference. It works like adding a second title to an existing binder.)
Convert	<pre>this_tuple = tuple(that_list) this_set = set(that_list)</pre>	※ Convert a list into a tuple (pass by value). ※ Convert a list into a set (pass by value).
Edit	<pre>Append haha.append("A") Insert haha.insert(index, value) Insert (Slice) haha[start : end]=["A", ...] Reverse haha.reverse()</pre>	※ Append value to end of collection. ※ Existing elements move backwards. ※ Replace [start : end] with ["A", ...]. Existing elements move backwards. ※ Reverse the elements of the list in place.
Other	Type Length Access(Slice) Delete Remove Pop Extend Concatenate Maximum Minimum Count Sort	

Tuples		
Create	<pre>the_tuple = (0, 6, 2, 5) the_tuple = 0, 6, 2, 5 tup = ("A",) the_tuple = tuple_1 + tuple_2 the_tuple = tuple*2 this_tuple = that_tuple the_tuple = tuple(formula for i in iterable if expression)</pre>	※ Create tuple with (,) round brackets (optional) and commas. ※ Without the comma, tup = ("A") creates a scalar variable. ※ Does not change dimension. ※ Still does not change dimension. ※ Duplicate tuples. ※ Create tuple from an iterable using comprehensions. Note that the tuple() function must be there.
Convert	<pre>this_list = list(that_tuple) this_set = set(that_tuple)</pre>	※ Convert a tuple into a list (pass by value). ※ Convert a tuple into a set (pass by value).
Other	Type Length Access(Slice) Delete Concatenate Maximum Minimum Count Sorted	

Sets			
Sets are collections of unique values (each specific value appears only once).			
Create	<pre>the_set = {0, 6, 2, 5} the_set = set() this_set = that_set the_set = {formula for i in iterable if expression}</pre>	※ Create set with {}, squiggly (curly) brackets and commas. ※ Create empty set (the_set = {} creates an empty dictionary). ※ Duplicate sets (pass by reference). ※ Create set from an iterable using comprehensions.	
Convert	<pre>this_list = list(that_set) this_tuple = tuple(that_set)</pre>	Edit	<pre>the_set.add("A")</pre>
			※ Add an element to the _set.
Other	Type Length Delete Remove Maximum Minimum Sorted		

Dictionaries				
Dictionaries contain data of key-value pairs. We are often referring to the keys when we mention a dictionary. If a specific key appears more than once, the ones before the last one would be covered. (Without warning!)				
Create	dict = { "M":0, "M":6, "D":2, "D":5 } this_dict = that_dict dict = {formula:formula for i in iterable if expression}		※ Create dictionary with { : , } curly brackets, columns, and commas. The key can be a string, int, float, boolean, or tuple. ※ Duplicate dictionary (pass by reference). ※ Create dictionary from an iterable using comprehensions.	
Convert	list_k = dict.keys() list_v = dict.values() list_d = list(dict) set_d = set(dict)	Edit	dict.clear() dict.update({key:value})	※ Clear dictionary. ※ Add element to dictionary.
Other	Type Length Access Delete Pop Maximum Minimum Sorted			

Data Types (Strings)

Strings can be applied in expressions (if substring in string). Strings can also be converted into booleans–True for not empty and False for empty. Strings are iterables—they can be iterated in for-loops. However, they cannot be edited through indices, nor can they change in place through methods (so you must use the assignment operator “=” in all cases).		
String Methods		
Related to ASCII Characters	ord(single_character) chr(decimal_reference)	※ The ord() method returns the ASCII-order of the character in decimal form.
Returning a Boolean	haha.isalnum() haha.isalpha() haha.isdigit() haha.islower() haha.isupper() haha.isspace() haha.endswith(value, start, end)	※ True if “all” characters in haha are: alphabetic or numeric, alphabetic, numeric, lowercase, uppercase. ※ True if haha[end-1]==value. (Optional start and end.)
Returning a String	haha.capitalize() haha.center(width,single_chr) haha.swapcase() haha.lower() haha.upper() replace(origin,replacement,n)	※ Make first character uppercase and the rest lowercase. ※ Make the old string centered by adding single_chr on both sides such that length of new string equals width. ※ Convert all character cases to the opposite, to lower, or to upper. ※ Replace the first n (parameter n is optional) appearance of the origin character with the replacement.
Returning a Number	haha.index(value, start, end) haha.find(value, start, end) haha.count(value, start, end)	※ Look for index of value’s first appearance (range is optional and does not affect the index-system). ※ The find() function returns (-1) if value not found. ※ Count how many times value appear in string.
Returning a List	haha.split(separator, maxsplit)	※ Default separator blank space. Maxsplit (optional) stops the process when there are enough pieces already.
Accepting an Iterable	haha.join(iterable)	※ Return a string of (element)(haha)(element)(haha)(element)(haha)...
Other	Max Min Sorted	

Data Files—Accessing Other Files

In programming, we sometimes want to access information from other files in the computer, and data files allow us to do so. We can open, close, edit, and read text files (.txt) through special methods. Here are some examples:			
Format 1 (only Editing or only Reading)			
Ideas (1) Explicitly state when to start and stop keeping track of the file or interpreter would only focus on the program file. (2) Give the file a “mode” to prepare interpreter for any related tasks later on. (3) Since programs are not familiar with files but variables, assign the information in the file to a variable. (4) Be aware of the cursor in the text file. All actions are related to it (E.G. where to start writing or reading).			
Process Open file. Set mode. Assign variable (and refer to the information-piece as the variable from now on). Do anything to the information-piece. Close information-piece.			

General Format	Interpretation		Mode	Method
A = open('name.txt', 'x') A.method() A.method() ...or... B = A.method() C = A.method() A.close()	Open or create file with such name, set it to mode x and assign it to variable A.		Read 'r'	B = A.read() #Read all lines and store it as string B.
				C = A.read(num) #Store whatever chars as string C.
				D = A.readline() #Store that line as string D.
				E = A.readlines() #Store each line in list E.
	Take out action.		Write 'w'	A.write("String\n") #Write and cover original text.
	Close information-piece.		Append 'a'	A.write("\nString") #Start a new line and write.

Format 2 (Another Way)		Format 3 (Editing alongside Reading)	
with open('name.txt', 'x') as A: for i in A: print(i) A.close()		A = open('name.txt', 'w+') #or 'a+' #or 'r+' A.write('Hello World!') A.seek(0, 0) #Move cursor to origin to start reading. print(A.read()) A.close()	
※ With-As-Statement: A = open('name.txt', 'x') can be written in the with-as-statement form.	※ For-Loop: for i in data_file: the i represents each line in the file.	※ Mode ('w+' as example): Informs the program that there are other tasks behind the writing task. But any other tasks (that is not writing) before the writing task would not be done because its existence was not hinted. ※ Seek: The cursor would be at the bottom and cannot read anything without the seek function.	

Exit Commands

quit() exit()	※ Both defined in site.py and only work if the site module is imported. This function should only be used in the interpreter. It should not be used in production code. (Production code means the code is being used by the intended audience in a real-world situation.)
import sys ... sys.exit()	※ Preferred way (but must import sys).

Output Formatting

Format 1 (Using Modulo Operator %)

General Format `print( "...%[f][w][.p]t..." %(value,value) )`

Idea Reserve a space of width “w” units. Take the corresponding value and round it to the precision “p” floating numbers; then print it as type “t” in the space reserved. Determine with flag “f” to either justify left or right.

	Examples–Code	Examples–Output	Reminders
Type s (String)	<code>print(“%5s %-4s” %(“Alb”,“ert”))</code> <code>print(“%5s %-4s” %(“Ho”,“ng”))</code>	<code>Alb ert</code> <code>Ho ng</code>	※ If flag contains nothing, justify right. ※ If flag is negative, justify left. ※ If the value contains more digits than the reserved width, the space expands. ※ Precision rounds up the number. ※ If no precision, the program would add zeros to make up 6 decimal numbers. ※ The precision is the number of digits you want “in the output.” ※ The number behind “E” or “e” tells you how to move the decimal point. ※ If %o, the program takes the value as a decimal number and prints it out as an octal (base 8) number.
Type d (Integer)	<code>print(“ 0%2d” %6)</code> <code>print(“ 0%2d” %625)</code>	<code>0 6</code> <code>0 6 2 5</code>	
Type f (Float)	<code>print(“%10.2f” %3.1415926535)</code> <code>print(“%10f” %2.71)</code>	<code>3 . 1 4</code> <code>2 . 7 1 0 0 0 0</code>	
Type E or e (Scientific Notation)	<code>print(“%10.3E” %1.61803)</code> <code>print(“%10.2e” %161803)</code> <code>print(“%10.1E” %0.00161803)</code>	<code>1 . 6 1 8 E + 0 0</code> <code>1 . 6 2 e + 0 5</code> <code>1 . 6 E - 0 3</code>	
Type o (Octal)	<code>print(“Number%3oB” %7)</code> <code>print(“Number%3oB” %8)</code>	<code>Number 7 B</code> <code>Number 1 0 B</code>	

Format 2 (Using {Curly Brackets} and f or F)

General Format `print( f“... {value:[w][.p]t}... {value:[w][.p]t}...” )`

or print(F“... {value:[w][.p]t}... {value:[w][.p]t}...”)

Idea When inside the print-round-brackets, there is a “f” or “F” in front of the quotation-marks, things inside the quotation-marks that are surrounded by curly brackets would be formatted. It works like the Modulo-Operator-Format but without flags—everything would be justified right in this case.

Other Tips

	Explanation	Examples
Escape Characters	Things that are in the quotation marks but not printed out are escape characters; they are just there for formatting. However, they can be printed out with the <code>repr()</code> function.	<code>print(“First Line\nSecond Line”)</code> <code>print(“First Column\tSecond Column”)</code> <code>print(repr(“Next Line \n and Tab \t”))</code> <code>print(“Unicode\u0040”) #prints@</code>
End	You can identify the end of your print statement in the print functions. Or by default, it goes to the next line.	<code>print(“Hello”, end=” ”)</code> <code>print(“World”)</code> <code>print(“New Line Here”)</code>
Number Base	When initialising a variable of numeric values, you can suggest the number base by adding 0[base] in front of the value.	<code>print(0b10001) #Binary, output 17.</code> <code>print(0o21) #Octal, output 17.</code> <code>print(0x11) #Hexadecimal, output 17.</code>
Quotation Marks	Output With “” : Collections, Repr Without “” : Single Strings	<code>print([“A”, “B”, “C”], repr>Hello\tWorld))</code> <code>print([“A”, “B”, “C”][0])</code>